codingninjas

Binary Search

Introduction to Searching Algorithms

Searching means to find out whether a particular element is present in a given sequence or not. There are commonly two types of searching techniques:

- Linear search (We have studied about this in **Arrays and Lists**)
- Binary search

In this module, we will be discussing binary search.

Binary Search

Binary Search is a searching algorithm for finding an element's position in a sorted array. In a nutshell, this search algorithm takes advantage of a collection of elements of are already sorted by ignoring half of the elements after just one comparison.

Prerequisite: Binary search has one pre-requisite; unlike the linear search where elements could be in any order, the array in **binary search** must be sorted,

The algorithm works as follows:

1. Let the element we are searching for, in the given array/list is X.
2. Compare X with the middle element in the array.
3. If X matches with the middle element, we return the middle index.
4. If X is greater than the middle element, then X can only lie in the right (greater) half subarray after the middle element, and then we apply the algorithm again for the right half.
5. If X is smaller than the middle element, then X must lie in the left (lower) half, this is because the array is sorted. So we apply the algorithm for the left half.

Example Run

- Let us consider the array to be:



- Let x = 4 be the element to be searched.
- Set two pointers **low** and **high** at the first and the last element respectively.
- Find the middle element **mid** of the array ie. **arr[(low+high)/2] = 6**.



- If **x == mid**, then return **mid**. Else, compare the element to be searched with **m**.
- If **x > mid**, compare x with the middle element of the elements on the right side of mid. This is done by setting low to **low = mid + 1**.
- Else, compare x with the middle element of the elements on the left side of mid. This is done by setting high to **high = mid-1**.



- Repeat these steps until low meets high. We found 4:




```
#Function to implement Binary Search Algorithm
def binarySearch(array, x, low, high):

    #Repeat until the pointers low and high meet each other
    while low <= high:
        mid = low + (high - low) // 2 #Middle Index
        if array[mid] == x: #Element Found
```

```
            return mid
        elif array[mid] < x: #x is on the right side
            low = mid + 1
        else: #x is on the left side
            high = mid - 1
    return -1  #Element is not found

array = [3, 4, 5, 6, 7, 8, 9]
x = 4

result = binarySearch(array, x, 0, len(array)-1)

if result != -1: #If element is found
    print("Element is present at index " + str(result))
else: #If element is not found
    print("Not found")
```

We will get the **output** of the above code as follows:

```
Element is present at index 1
```

Advantages of Binary search:

- This searching technique is faster and easier to implement.
- Requires no extra space.
- Reduces the time complexity of the program to a greater extent. (The term **time complexity** might be new to you, you will get to understand this when you will be studying algorithmic analysis. For now, just consider it as the time taken by a particular algorithm in its execution, and time complexity is determined by the number of operations that are performed by that algorithm i.e. time complexity is directly proportional to the number of operations in the program).

Problem Statement: Finding the Lower Bound

An algorithm to find the lower bound of a target value in a sorted array using binary search. The lower bound of a target value is defined as the index of the first element in the array that is not less than the target value. If all elements are less than the target, the function returns the length of the array. Here is a detailed step-by-step explanation of the approach:

1. **Initialization:**
    - **start** is initialized to 0.
    - **end** is initialized to the last index of the array (len(arr) - 1).
    - **ans** is initialized to the length of the array (len(arr)), representing the default case where no element is found.
2. **Binary Search Loop:**
    - The loop runs as long as start is less than or equal to end.
    - Mid Calculation:
        - Calculate mid as the integer division of start and end (i.e., (start + end) // 2).
3. **Condition Checking:**
    - If the element at the mid index is greater than or equal to the target:
        - Update ans to mid (since mid is a candidate index for the lower bound).
        - Move the end pointer to mid-1 to search for a potentially smaller index on the left side.
    - If the element at the mid-index is less than the target:
        - Move the start pointer to mid + 1 to search for the target in the right half.
4. **Result:**
    - The loop will terminate when start is greater than end.
    - The value of ans at this point will be the index of the first element that is not less than the target value.
    - If no such element exists, ans will be equal to the length of the array, indicating that all elements are less than the target.

```
def lower_bound(arr, target):
    start = 0
    end = len(arr) - 1
```

```
    ans = len(arr)
    while start <= end:
        mid = (start + end) // 2
        if arr[mid] >= target:
            ans = mid # store the candidate ans
            # let's fund even better ans
            end = mid - 1
        else:
            start = mid + 1

    return ans
```

Problem Statement: Find the Square root

An algorithm to find the integer part of the square root of a given number “n” using a binary search approach. Here is a detailed step-by-step explanation of the approach:

1. **Initialization:**
    - **start** is initialized to 1.
    - **end** is initialized to nnn.
    - **ans** is initialized to end, which will store the probable answer.
2. **Binary Search Loop:**
    - The loop runs as long as start is less than or equal to end.
    - Mid Calculation:
        - Calculate mid as the integer division of start and end (i.e., (start + end) // 2).
3. **Condition Checking:**
    - Check if mid * mid is less than or equal to n:
        - If true, this means mid is a potential candidate for the square root of n. Therefore:
            - Update ans to mid (since mid is a probable answer).
            - Move the start pointer to mid + 1 to search for a potentially larger value.
    - If mid * mid is greater than n:
        - This means mid is too large to be the square root of n. Therefore:
            - Move the end pointer to mid-1 to search for a smaller value.

4. **Result:**
    - The loop will terminate when start is greater than end.
    - The value of ans at this point will be the largest integer whose square is less than or equal to nnn, which is the integer part of the square root of nnn.

```
def squareroot(n):
    # define the search space
    start = 1
    end = n
    ans = end
    # till the time search space is available
    while start <= end:
        mid = (start + end) // 2
        if mid*mid <= n:
            # We have found a probable ans and we can find
something better on the right
            # discard left
            start = mid + 1

            ans = mid # storing the probable ans
        else:
            # we cannot find anything better on right side,
so discard the right side
            end = mid - 1

    return ans
```

Problem Statement: Finding an Element in a 2D Matrix

An algorithm to find a target element in a 2D matrix, where each row and each column of the matrix is sorted in ascending order. This is done using a binary search technique by treating the 2D matrix as a 1D array. Here is a detailed step-by-step explanation of the approach:

1. **Initialization:**
    - m is initialized to the number of rows in the matrix.
    - n is initialized to the number of columns in the matrix.
    - start is initialized to 0 (representing the first element in the flattened 1D view of the matrix).

- end is initialized to m * n - 1 (representing the last element in the flattened 1D view of the matrix).

2. **Binary Search Loop:**
   - The loop runs as long as start is less than or equal to end.
   - Mid Calculation:
     - Calculate mid as the integer division of start and end (i.e., start + (end - start) // 2).
3. **Convert 1D Index to 2D Indices:**
   - Calculate the row index as mid // n.
   - Calculate the column index as mid % n.
4. **Element Check:**
   - Retrieve the element at the calculated row and column indices.
   - If the element is equal to the target, return True.
   - If the element is less than the target, move the start pointer to mid + 1 to search in the right half.
   - If the element is greater than the target, move the end pointer to mid-1 to search in the left half.
5. **Result:**
   - The loop will terminate when start is greater than end.
   - If the target is not found, return False.

```
def find_in_2d(grid, target):
    m = len(grid)
    n = len(grid[0])
    start = 0
    end = m*n-1

    while start <= end:
        mid = start + (end - start) // 2

        row = mid // n
        col = mid%n
        element = grid[row][col]

        if element == target:
            return True
        if element < target:
            start = mid + 1
        else:
```

```
            end = mid - 1

    return False
```

Problem Statement: Finding an Element in a Sorted Rotated Array

An algorithm to find a target element in a sorted and rotated array using a modified binary search. Here's a detailed step-by-step explanation of the approach:

1. **Initialization:**
    - **start** is initialized to 0.
    - **end** is initialized to the last index of the array (len(li) - 1).
2. **Binary Search Loop:**
    - The loop runs as long as start is less than or equal to end.
    - Mid Calculation:
        - Calculate mid as the integer division of start and end (i.e., (start + end) // 2).
3. **Element Check:**
    - If the element at the mid index is equal to the target, return mid as the index of the target.

4. **Determine Which Part is Sorted:**
    - Check if the mid element is in the lower (rotated) part or the upper (non-rotated) part of the array:
        - Lower Curve Check:
            - If li[start] > li[mid]:
                - The array is rotated, and the mid element is on the lower curve.
                - Target in Right Half:
                    - If the target is between mid and end (li[mid] <= target <= li[end]):
                        - Discard the left part by setting start = mid + 1.
                    - Target in Left Half:
                        - Otherwise, discard the right part by setting end = mid - 1.
        - Upper Curve Check:
            - If li[start] <= li[mid]:

- The array is not rotated in the range between start and mid, indicating the mid element is on the upper curve.
- Target in Left Half:
  - If the target is between start and mid (li[start] <= target <= li[mid]):
    - Discard the right part by setting end = mid - 1.
  - Target in Right Half:
    - Otherwise, discard the left part by setting start = mid + 1.

5. **Result:**
   - The loop will terminate when start is greater than end.
   - If the target is not found, return -1.

```
def find_in_sorted_rotated(li, target):
    # define search space
    start = 0
    end = len(li) - 1

    while start <= end:
        # calculate mid
        mid = (start + end) // 2

        if li[mid] == target:
            return mid

        # decide if mid is on upper curse or lower curve
        if li[start] > li[mid]:
            # we are on lower curve
            if li[mid] <= target <= li[end]:
                # discard left part
                start = mid + 1
            else:
                #discard right
                end = mid - 1
        else:
```

codingninjas

```
            # we are on the upper curve
            if li[start] <= target <= li[mid]:
                # discard right
                end = mid - 1
            else:
                # discard left
                start = mid + 1

    return -1
```